

Московский авиационный институт
(Национальный исследовательский университет)
Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №2
по курсу «Теория сложности алгоритмов»
Вариант №2

Выполнила студентка: Бондарева Е. Е.

Группа: М8О-109СВ-25

Преподаватель: Рассказова В. А.

Дата: ____17.03.2026____

Оценка: _____

Москва, 2026

Задание

Для заданного КС-языка B над алфавитом $\Sigma = \{0, 1\}$:

1. Построить диаграмму состояний МП-автомата, распознающего A .
2. Реализовать данный МП-автомат в виде программы, которая для произвольной входной строки w должна выводить историю вычислений МП-автомата на ней в виде последовательности состояний и содержимого стека памяти.

$B = \{w: w \text{ содержит 1 больше, чем } 0\}$;

Теоретическая часть

Опр. 1.1. Контекстно-свободной грамматикой (КС-грамматикой) называется четверка вида $G = V, \Sigma, R, S$,

где V -- множество переменных грамматики (буквенные символы),

Σ -- алфавит грамматики,

$R \subset V \times \Sigma \cup V^*$ -- множество правил грамматики,

S -- начальная переменная.

Опр. 1.2. Говорят, что в грамматике G из строки вида uAv выводится строка uwv (обозначается как $uAv \Rightarrow uwv$), если существует правило $(A \rightarrow w) \in R$. Для краткости, в случае применения последовательности правил используют запись вида $u \Rightarrow^* v$.

Опр.1.3. Языком грамматики G называется множество строк, которые выводятся из начальной переменной $L G = \omega: S \Rightarrow^* \omega$.

Опр.1.4. Язык A называется контекстно-свободным (КС-язык) если существует КС-грамматика G такая, что $A = L G$.

Опр. 2.1. Автоматом с магазинной (МП-автомат, МПА) называется шестерка вида $M = Q, \Sigma, \Gamma, \delta, q_0, F$,

где Q -- множество состояний,

Σ -- алфавит,

Γ -- алфавит стека,

$\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow P(Q \times \Gamma_\varepsilon)$ -- переходная функция (по виду переходной функции ясно, что по умолчанию МПА является недетерминированным),
 q_0 -- начальное состояние,
 F -- множество допускающих состояний.

Опр. 2.2. Говорят, что МПА M допускает строку ω если существуют последовательности $r_1, \dots, r_{n+1} \in Q$ и $s_1, \dots, s_{n+1} \in \Gamma_\varepsilon$ такие, что

- 1) $r_1 = q_0$,
- 2) $(r_{i+1}, b) \in \delta(r_i, x_i, a)$, где $s_i = at$, $s_{i+1} = bt$, $a, b \in \Gamma_\varepsilon$, $t \in \Gamma_\varepsilon^*$,
- 3) $r_{n+1} \in F$.

Опр.2.3. Языком МПА называется множество строк, которые он допускает $L(M)$. Если некоторый язык A такой, что $A=L(M)$, то говорят, что M распознает A .

Теорема 2.1. Если язык A является КС-языком, то существует МПА такой, что $A=L(G)$.

Доказательство. По условию теоремы A -- это КС-язык. Значит, по опр. 1.4. существует эквивалентная ему КС-грамматика G . Построим МПА, эквивалентный этой грамматике по следующему алгоритму:

- 1) поместить в стек S и $\#$,
- 2) повторять
 - 2.1) если на стеке $A \in V$ то недетерминированно заменить на все правила из множества R ,
 - 2.2) если на стеке $a \in \Sigma$ то удалить его, считать символ входной строки и сравнить: если не совпадают, то отвергнуть строку,
 - 2.3) если на стеке $\#$, то перейти в допускающее состояние.

По построению МПА будет допускать только те строки, которые выводятся в грамматике G .

Теорема 2.2. Язык A КС-язык тогда и только тогда, когда существует эквивалентный ему МПА.

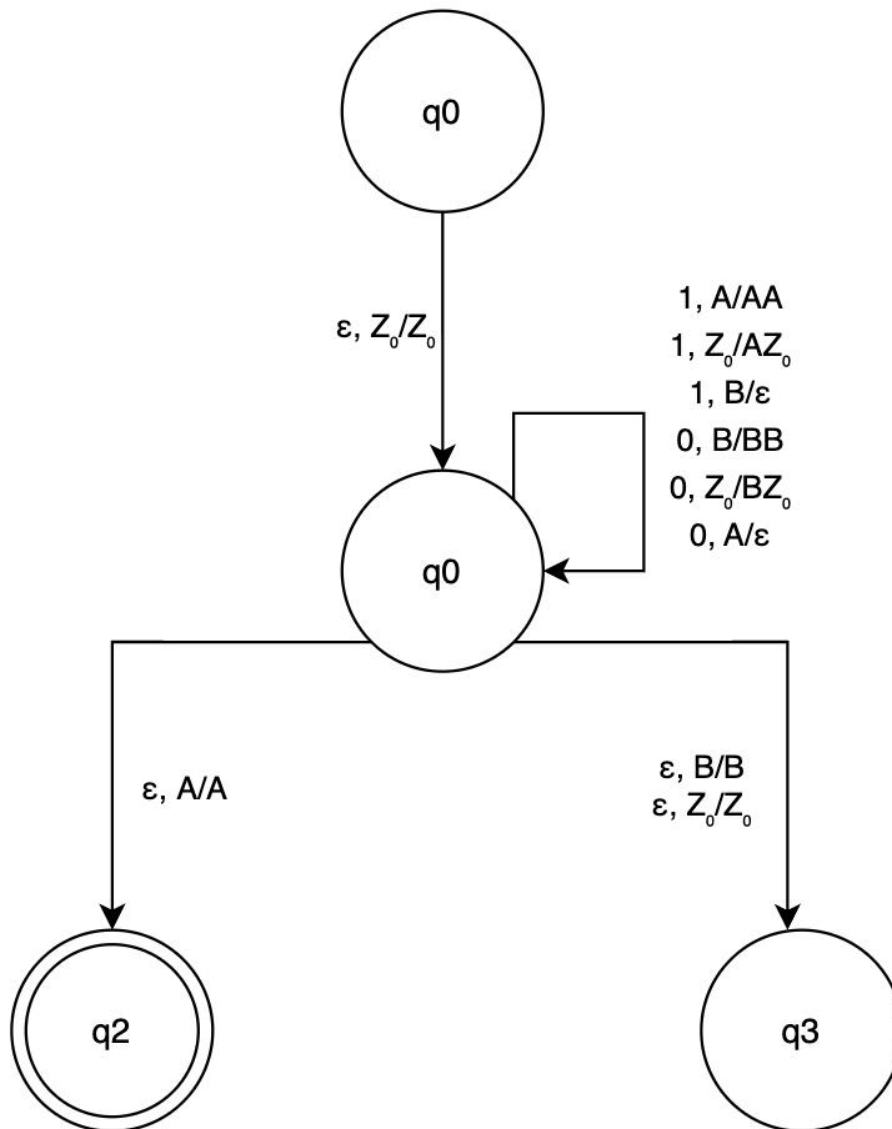
Теореме 2.3. Для любого ДКА существует эквивалентная КС-грамматика (а, следовательно, и эквивалентный МПА -- таким образом, любой регулярный язык является КС-языком, но обратное неверно).

Доказательство. Рассмотрим произвольный ДКА и построим эквивалентную ему КС-грамматику по следующему алгоритму:

- 1) для каждого $q_i \in Q$ добавить переменную $V_i \in V$
- 2) для каждого $q_i \in F$ добавить правило вида $V_i \rightarrow \varepsilon$
- 3) для всех $\delta(q_i, a) = q_j$ добавить правило $V_i \rightarrow aV_j$
- 4) положить $S = V_0$

По построению $L(M) = L(G)$.

Диаграмма



Код программы

```
stack = []
input_str = input()

for i in input_str:
    if i not in ('1', '0'):
        print(f"Встретили некорректный символ {i}")
        exit(0)

cntr = 0
print(f"Шаг {cntr}")
print("Состояние: q0")
print(f"Стек: {stack}\n")

for i in input_str:
    cntr += 1
    if i == '1':
        if stack.__len__() == 0 or stack[-1] == "A":
            stack.append("A")
        else:
            del stack[-1]
    elif i == '0':
        if stack.__len__() == 0 or stack[-1] == "B":
            stack.append("B")
        else:
            del stack[-1]

    print(f"Шаг {cntr}")
    if stack.__len__() == 0:
        print("Состояние: q0")
    elif stack[-1] == "A":
        print("Состояние: q1")
    else:
        print("Состояние: q2")
    print(f"Стек: {stack}\n")

if stack.__len__() == 0 or stack[-1] == "A":
    print("Допускается")
else:
    print("Не допускается")
```

Тесты

1) Входное число: 123456789

Вывод: Встретили некорректный символ 2

2) Входное число: 10101010

Вывод:

Шаг 0

Состояние: q0

Стек: []

Шаг 1

Состояние: q1

Стек: ['A']

Шаг 2

Состояние: q0

Стек: []

Шаг 3

Состояние: q1

Стек: ['A']

Шаг 4

Состояние: q0

Стек: []

Шаг 5

Состояние: q1

Стек: ['A']

Шаг 6

Состояние: q0

Стек: []

Шаг 7

Состояние: q1

Стек: ['A']

Шаг 8

Состояние: q0

Стек: []

Не допускается

3) Входное число: 00000

Вывод:

Шаг 0

Состояние: q0

Стек: []

Шаг 1

Состояние: q2

Стек: ['В']

Шаг 2

Состояние: q2

Стек: ['В', 'В']

Шаг 3

Состояние: q2

Стек: ['В', 'В', 'В']

Шаг 4

Состояние: q2

Стек: ['В', 'В', 'В', 'В']

Шаг 5

Состояние: q2

Стек: ['В', 'В', 'В', 'В', 'В']

Не допускается

4) Входное число: 11111111000

Вывод:

Шаг 0

Состояние: q0

Стек: []

Шаг 1

Состояние: q1

Стек: ['А']

Шаг 2

Состояние: q1

Стек: ['А', 'А']

Шаг 3

Состояние: q1

Стек: ['А', 'А', 'А']

Шаг 4

Состояние: q1

Стек: ['А', 'А', 'А', 'А']

Шаг 5

Состояние: q1

Стек: ['А', 'А', 'А', 'А', 'А']

Шаг 6

Состояние: q1

Стек: ['А', 'А', 'А', 'А', 'А', 'А']

Шаг 7

Состояние: q1

Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 8

Состояние: q1

Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 9

Состояние: q1

Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 10

Состояние: q1

Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 11

Состояние: q1

Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 12

Состояние: q1

Стек: ['A', 'A', 'A', 'A', 'A', 'A']

Допускается

5) Входное число: 1111111111

Вывод:

Шаг 0

Состояние: q0

Стек: []

Шаг 1

Состояние: q1

Стек: ['A']

Шаг 2

Состояние: q1

Стек: ['A', 'A']

Шаг 3

Состояние: q1

Стек: ['A', 'A', 'A']

Шаг 4

Состояние: q1

Стек: ['A', 'A', 'A', 'A']

Шаг 5

Состояние: q1
Стек: ['A', 'A', 'A', 'A', 'A']

Шаг 6
Состояние: q1
Стек: ['A', 'A', 'A', 'A', 'A', 'A']

Шаг 7
Состояние: q1
Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 8
Состояние: q1
Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 9
Состояние: q1
Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']

Шаг 10
Состояние: q1
Стек: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A']

Допускается

6) Входное число: *пустая строка*

Вывод:

Шаг 0
Состояние: q0
Стек: []

Не допускается

Вывод

В результате выполнения лабораторной работы для заданного КС-языка В над алфавитом, содержащим только нули и единицы, была построена диаграмма состояний МП-автомата, распознающего А, а также реализован МП-автомат в виде программы, которая для произвольной входной строки w должна выводить историю вычислений МП-автомата на ней в виде последовательности состояний и содержимого стека памяти при условии, что входная строка содержит единиц больше, чем нулей.